SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305 Name:T Gnaneswari Reg no:192372183

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS: **Total Marks: 25**

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.

4.An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5.A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I T Gnaneswari(192372183) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:

T Gnaneswari

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect

WEB SERVICES: DESIGN, SCALABILITY, SECURITY, API INTEGRATION & FAULT TOLERANCE

1. Analysis of REST and SOAP and Selection of an Appropriate Solution

A company building one backend for both mobile and web clients must select a service style that supports fast responses, efficient bandwidth usage, horizontal growth and secure access. REST and SOAP can both provide web services, but they differ in communication model, message format and operational overhead.

REST Approach

REST (Representational State Transfer) models application data as resources identified by URLs. Clients use standard HTTP methods such as GET, POST, PUT/PATCH and DELETE. JSON is commonly used because it is compact and directly supported by browsers and mobile frameworks. REST is normally stateless, meaning the server does not need to remember client session state between API requests.

SOAP Approach

SOAP is a standardized XML messaging protocol. A SOAP message uses an Envelope, optional Header and Body. Services are commonly described through WSDL, while XML Schema defines data types. SOAP offers mature enterprise standards including message-level security and formal fault messages, but XML serialization and parsing generally add more payload and processing overhead.

Factor	REST	SOAP	Impact	Preferred
Data format	Usually JSON	XML	JSON is typically smaller	REST
Performance	Lightweight	More XML processing	Important for mobile	REST

Scalability	Natural stateless design	Can scale, but often heavier	Easy horizontal scaling	REST
Contract	OpenAPI commonly used	WSDL is formal	SOAP has stricter contract	Depends
Security	HTTPS + OAuth/JWT	HTTPS + WS-Security	Both can be secure	REST for this case
Caching	HTTP caching friendly	Less natural	Reduces server load	REST

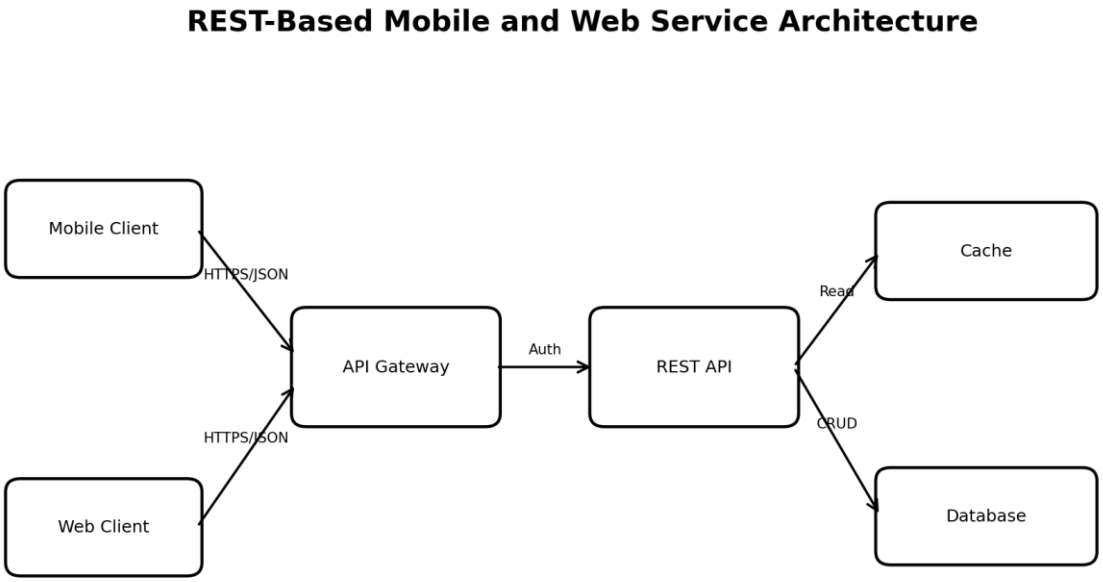

Figure 1. Recommended REST architecture for mobile and web applications.

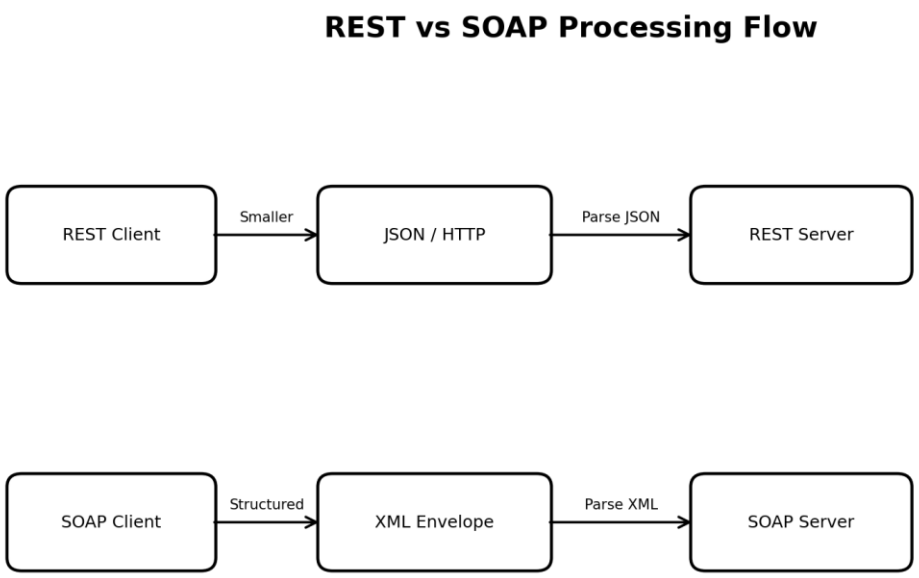

Figure 2. Simplified comparison of REST and SOAP message processing.

Recommended Design

The recommended solution is a REST API using JSON over HTTPS. Both mobile and web clients call an API gateway. The gateway performs authentication, authorization, throttling and request routing. Stateless REST service instances execute business logic and access the cache or database. This design makes it easy to add more service instances when traffic increases.

Justification

- Performance: smaller JSON payloads and simpler parsing reduce bandwidth and CPU cost for typical mobile/web operations.
- Scalability: stateless requests can be distributed across many server instances through a load balancer.
- Security: HTTPS/TLS protects data in transit; OAuth 2.0, short-lived access tokens, role-based authorization and rate limits protect the API.
- Maintainability: a consistent resource model and standard HTTP semantics make APIs easier to develop and test.
- Interoperability: virtually every web and mobile platform can consume HTTP/JSON APIs.

SOAP would still be a reasonable alternative if the organization required a strict WSDL contract, legacy SOAP integration, message-level signatures/encryption, or specific WS-* enterprise standards. For the stated mobile/web scenario, REST gives the better overall balance.

2. Scalable Service Architecture for a Large Online Platform

When many users access a platform simultaneously, a single application server and single database become bottlenecks and single points of failure. Scalability must therefore be designed across the network edge, application layer, background processing and data layer.

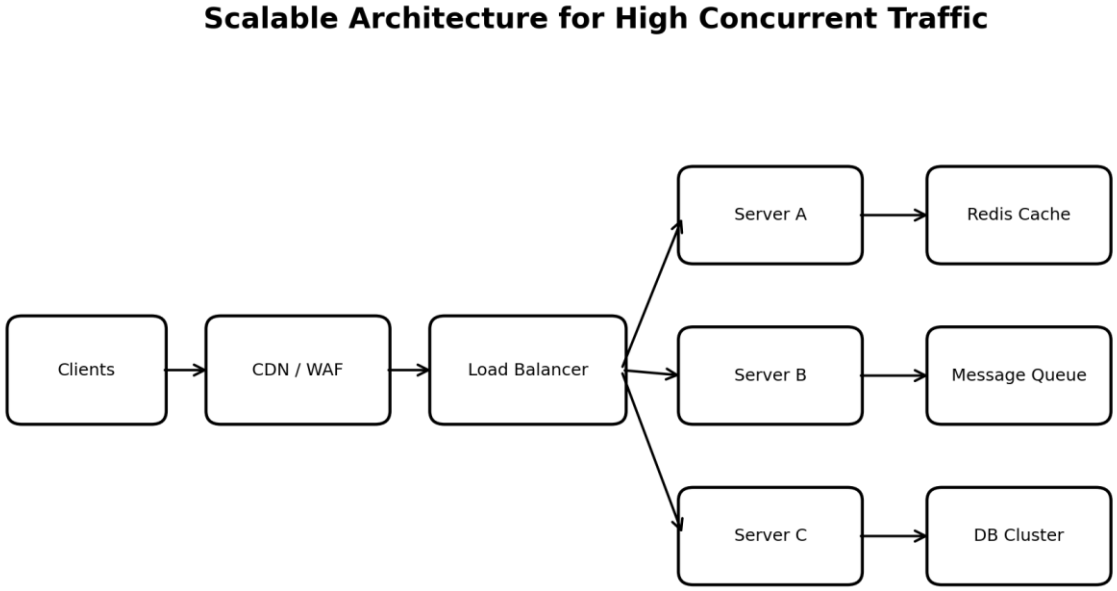

Figure 3. Scalable architecture with CDN/WAF, load balancing, multiple servers, cache, queue and database cluster.

Component Interaction

- Clients: browser and mobile clients generate HTTPS requests.
- CDN/WAF: static content can be served close to users, while the web application firewall filters common malicious traffic.
- Load balancer: distributes requests among healthy application instances.
- Application servers: stateless instances execute business logic. More instances can be added horizontally.
- Cache: frequently read data and temporary computed results can be stored in Redis or a similar in-memory cache.
- Message queue: slow tasks such as email, report generation or media processing are moved away from the synchronous request path.

- Database cluster: primary/replica or other distributed data strategies improve read capacity and availability.

Request Flow

1. Client sends an HTTPS request.
2. CDN/WAF handles edge delivery and filtering.
3. Load balancer selects a healthy server.
4. Server checks cache for frequently requested information.
5. On a cache miss, the service queries the database.
6. Long-running work is published to a queue and processed by workers.
7. Response is returned to the client while monitoring records latency and errors.

Alternative Design: Microservices

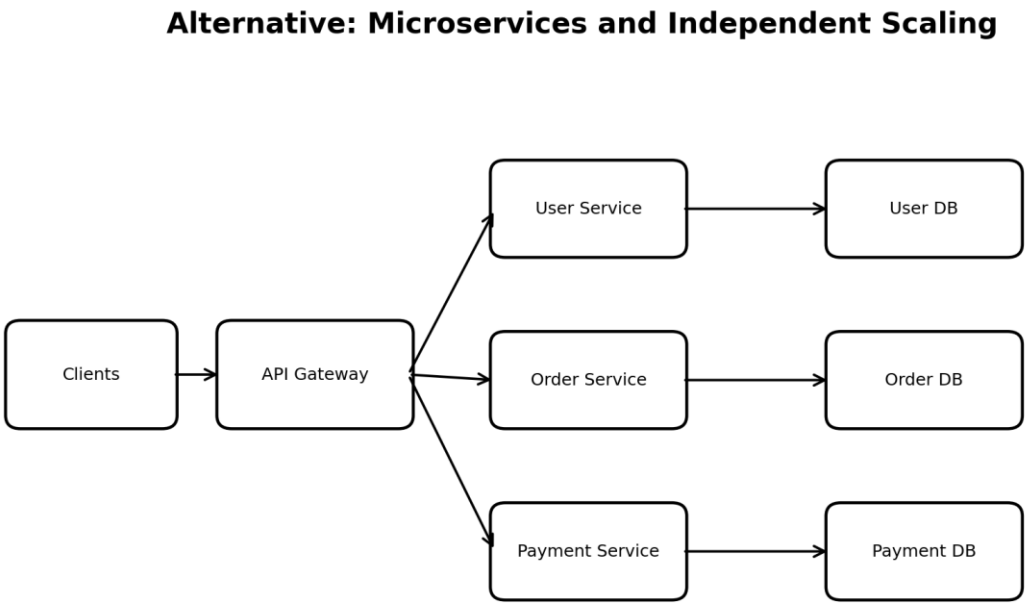

Figure 4. Alternative microservices design with independently scalable services and data stores.

A large monolithic service can be decomposed into business capabilities such as User, Order and Payment services. An API gateway exposes a unified entry point. Each service can be deployed and scaled independently. A sale may require ten Order instances while the User service may need only three. This avoids scaling the entire application for one hot feature.

Scalability Improvements

- Horizontal autoscaling based on CPU, memory, request rate or queue depth.
- CDN caching for images, JavaScript, CSS and other static content.
- Application caching to reduce repeated database reads.
- Database read replicas for read-heavy workloads and sharding only when a single database can no longer meet scale.
- Asynchronous queues for burst absorption and long-running work.
- Health checks and redundancy so failed instances are removed automatically.
- Connection pooling to avoid repeatedly creating database connections.
- Rate limiting and backpressure to protect downstream systems during overload.

The alternative microservices architecture improves independent scalability and fault isolation, but it adds operational complexity. Therefore, a modular monolith with horizontal scaling may be best initially, followed by selective service extraction when traffic and organizational scale justify it.

3. Secure Communication Protocol for a Banking Application

Banking systems transmit highly sensitive information and transaction instructions. The communication channel must prevent eavesdropping, tampering and server impersonation while still providing acceptable performance and dependable delivery.

Protocol	Security	Performance	Suitability
HTTP	No transport encryption	Fast but unsafe	Not suitable
HTTPS/TLS	Encryption, integrity, server authentication	Small modern TLS overhead	Best general choice
SOAP over HTTPS	TLS plus optional SOAP security standards	More XML overhead	Useful for enterprise SOAP integration
WebSocket over TLS (WSS)	Encrypted persistent connection	Efficient for real-time streams	Specialized, not a replacement for normal banking API calls

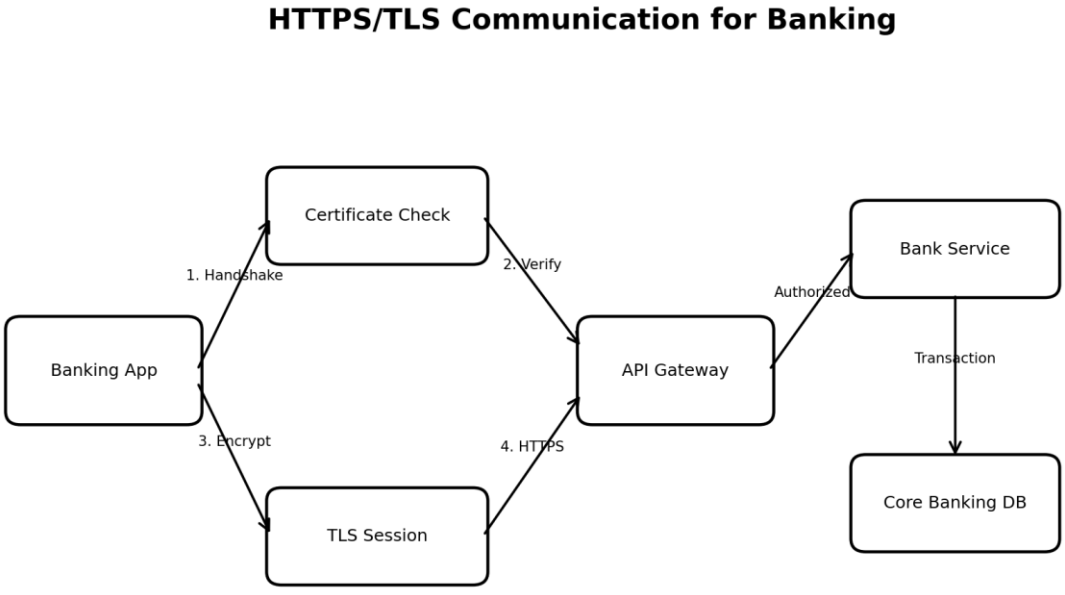

Figure 5. Banking communication secured with HTTPS/TLS.

Selected Protocol: HTTPS with TLS

HTTPS is HTTP transported through TLS. During the TLS handshake the server presents a certificate, the client verifies the certificate and both sides establish cryptographic session keys. Subsequent application requests and responses are encrypted and integrity-protected.

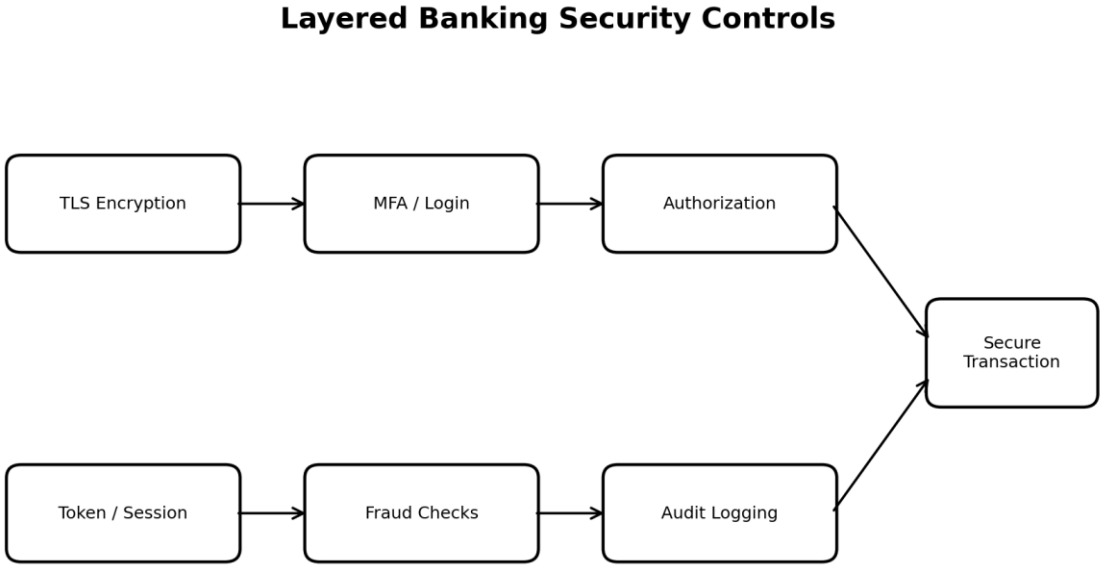

Figure 6. Layered controls surrounding a secure banking transaction.

Security and Reliability Justification

- Confidentiality: encrypted traffic protects credentials and transaction data from passive interception.
- Integrity: TLS detects modification of protected records in transit.
- Authentication: certificate validation helps the client confirm it is communicating with the legitimate banking endpoint.
- Application authentication: MFA, secure sessions/tokens and transaction authorization provide controls beyond transport encryption.
- Reliability: APIs should use timeouts, idempotency keys and safe retry rules so a network failure does not accidentally duplicate a transfer.
- Auditability: transaction IDs, security events and access logs provide traceability.
- Performance: modern TLS session reuse and HTTP/2 can reduce connection and request overhead.

For server-to-server banking integrations, mutual TLS can authenticate both parties. The application should also use secure key management, least-privilege authorization, fraud detection, input

validation and strong monitoring. HTTPS/TLS is therefore the most suitable base protocol, with additional banking controls layered above it.

4. API Integration Strategy for Payment Gateways or Map Services

Third-party APIs introduce an external dependency into the application. A good integration design protects credentials, validates all input and output, prevents duplicate operations, handles provider limits and isolates provider failures from the main user experience.

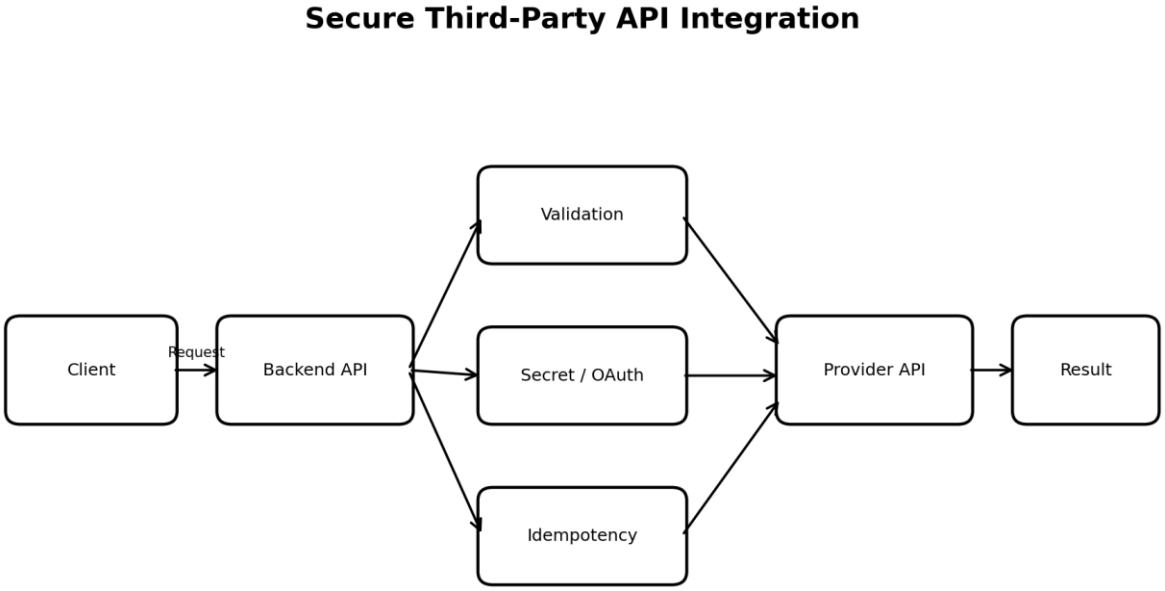

Figure 7. Secure request, authentication and response flow for third-party API integration.

Integration Strategy

1. The client sends a business request to the application's own backend rather than directly using private provider credentials.
2. The backend authenticates the user and validates required fields, amount, coordinates or other domain rules.
3. The integration layer obtains the required API key, OAuth token or signed credential from secure server-side configuration.
4. The backend sends the provider request over HTTPS with a unique request/correlation identifier.
5. The provider response is checked for HTTP status, expected schema, provider-specific error codes and business consistency.
6. The application stores only the required result and returns a normalized response to the client.
7. Metrics and logs record latency, failures and rate-limit responses without logging sensitive secrets.

Authentication Methods

Method	Typical use	Important control
API key	Simple server-to-server APIs	Keep key on backend; rotate it
OAuth 2.0	Delegated/user-authorized access	Use scoped, expiring tokens
Signed request / HMAC	High-integrity provider calls	Protect secret and verify timestamp/signature
mTLS	High-assurance B2B integration	Manage client certificates securely

Alternative: Asynchronous Queue + Webhook

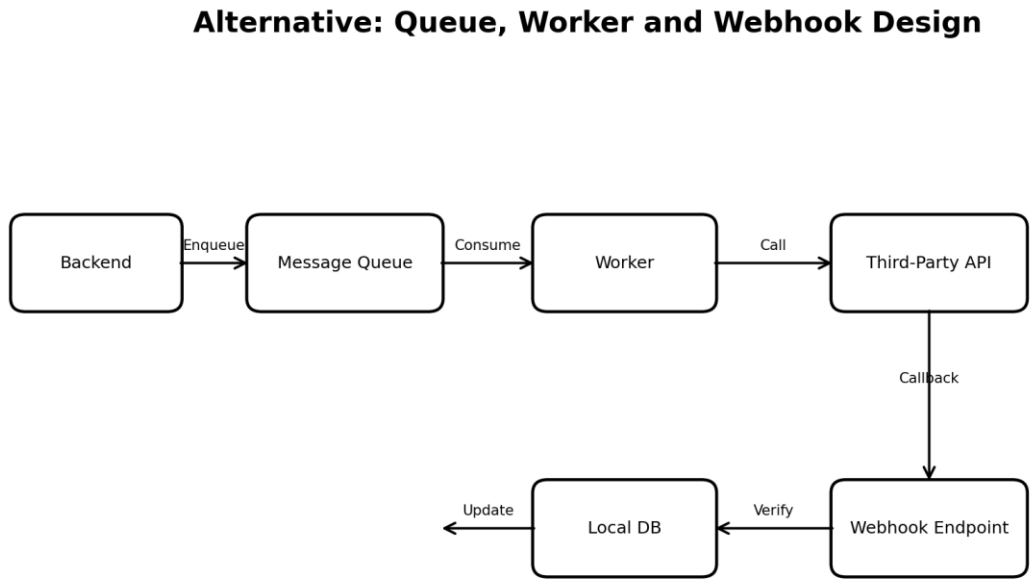

Figure 8. Queue-based integration with worker processing and verified webhook callback.

For operations that do not need an immediate final result, the backend can enqueue work. A worker consumes the message and calls the provider. If the provider is temporarily unavailable, the worker retries using exponential backoff and jitter. Payment providers can later send a signed webhook; the application verifies the signature and updates local state.

- Idempotency keys prevent duplicate payment creation when a request is retried.
- Timeouts prevent provider calls from occupying application threads indefinitely.
- Circuit breakers stop repeated calls during a provider outage.
- Rate-limit awareness prevents the application from exceeding provider quotas.

- Fallbacks can use cached map data or temporarily disable nonessential provider features.
- Provider adapters isolate third-party-specific formats from the rest of the application.

The asynchronous alternative improves burst handling and reliability because external slowness does not block the entire user-facing request path. However, the user interface must clearly represent pending states for operations that complete later.

5. Error Handling and Fault Tolerance for a Web Service

Frequent errors should not be handled with a generic 500 response. The system needs a layered mechanism that detects bad requests early, prevents cascading failures, records diagnostics and provides users with stable, understandable responses.

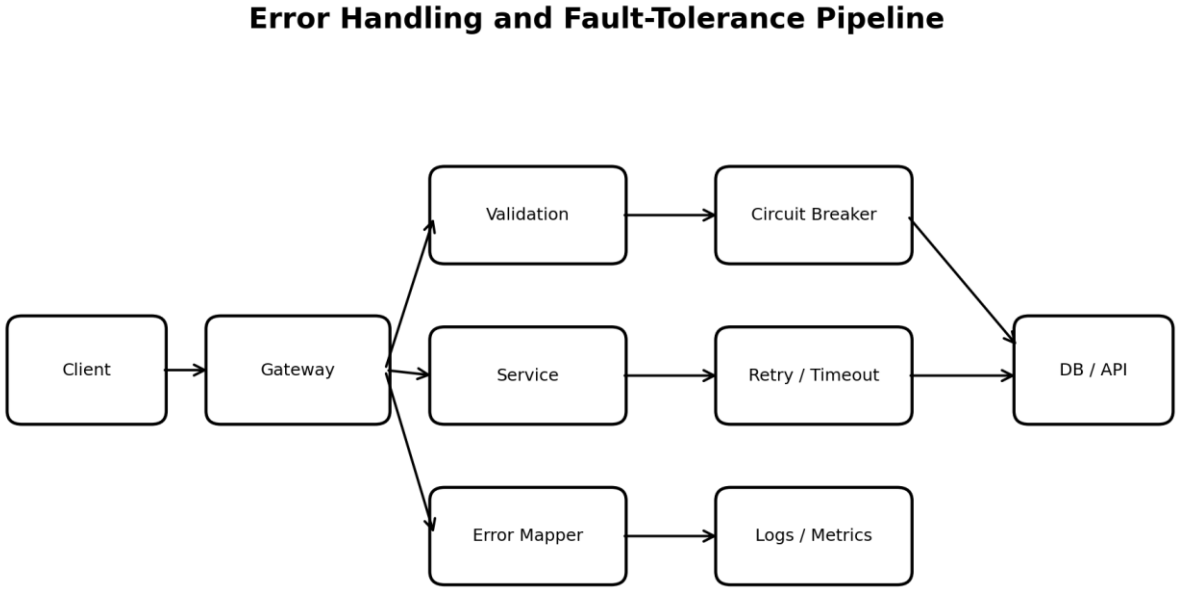

Figure 9. Layered error handling, retries, circuit breaker, logging and dependency access.

Error Classification

Status	Meaning	Example	Client behavior
400	Bad request	Invalid field or JSON	Correct request
401	Unauthenticated	Missing/expired login	Authenticate again
403	Forbidden	Insufficient role	Do not retry blindly
404	Not found	Unknown resource	Check identifier
409	Conflict	Duplicate/state conflict	Refresh or resolve conflict
429	Too many requests	Rate limit exceeded	Retry after delay
500	Internal error	Unexpected server exception	Show safe message

503	Unavailable	Temporary dependency/server outage	Retry safely later

Structured Error Response

```
{
 "error": {
  "code": "PAYMENT_PROVIDER_TIMEOUT",
  "message": "The service is temporarily unavailable. Please try again.",
  "requestId": "REQ-8F21A"
 }
}
```

The public response should not reveal stack traces, database statements, internal hostnames or secrets. The request ID links the user's failure to protected server logs and distributed traces.

Fault-Tolerance Mechanisms

- Validation: reject malformed or impossible input before expensive processing.
- Timeouts: bound how long the service waits for databases and external APIs.
- Retry with exponential backoff: retry only transient failures and only when the operation is safe or protected by idempotency.
- Circuit breaker: stop sending calls to a dependency that is repeatedly failing.
- Bulkheads: isolate resource pools so failure in one dependency does not consume all application threads or connections.
- Fallback/cache: return safe cached information or reduced functionality when appropriate.
- Load balancing and redundant instances: route traffic away from unhealthy servers.
- Queues: persist asynchronous work and smooth sudden traffic spikes.
- Dead-letter queue: isolate repeatedly failing asynchronous messages for later investigation.
- Observability: metrics, centralized logs, traces and alerts identify error patterns quickly.

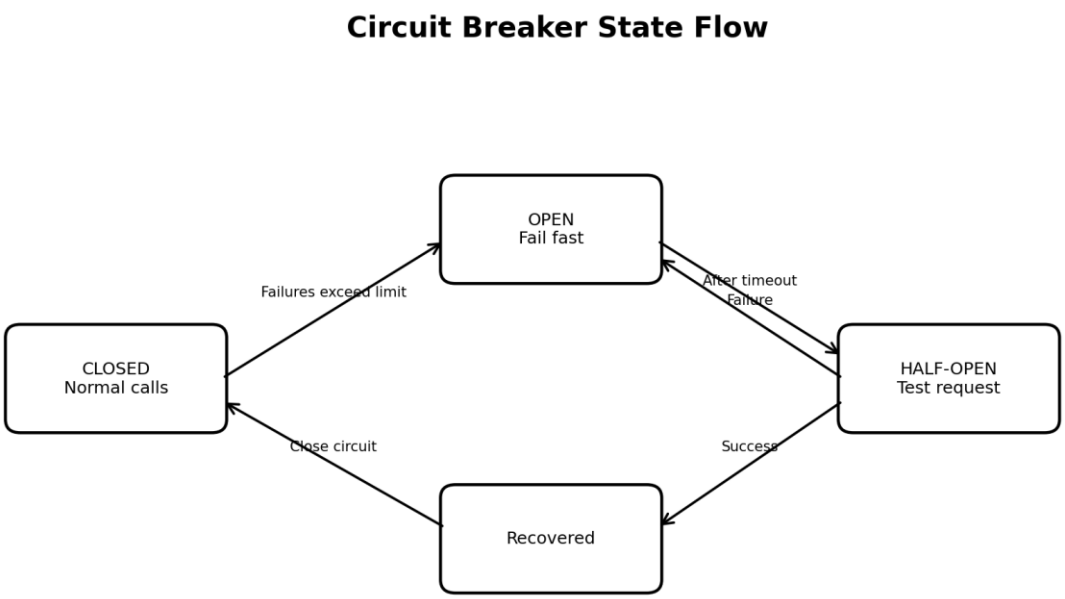

Figure 10. Circuit breaker states used to prevent cascading failures.

Smooth User Experience

Users should receive clear, actionable messages and stable application behavior. A temporary provider failure can display 'Payment confirmation is taking longer than expected' rather than an internal exception. The UI can preserve entered data, avoid duplicate submissions, show retry or pending states and continue offering unaffected features.

Alternative Reliability Strategies

For critical systems, reliability can be improved further through multi-zone deployment, database failover, automated health checks, backup/restore testing, chaos/failure testing, graceful degradation and service-level objectives. The key principle is to assume individual components will eventually fail and design the system so that one failure does not become a total outage.

Overall Conclusion

For modern mobile and web applications, REST over HTTPS is generally the strongest default because it is lightweight and easy to scale. High-traffic platforms should use horizontal scaling, load balancing, caching, queues and resilient data architecture. Banking systems require TLS plus strong application-level authentication and transaction controls. Third-party APIs should be isolated through secure backend integration, with idempotency, retries and webhooks where appropriate. Finally, reliable web services require structured errors, timeouts, circuit breakers, redundancy and observability so failures are contained and users receive a smooth experience.

Practical Examples for All Five Questions

The following examples can be written after the corresponding theoretical answers to demonstrate how each architecture works in a real application.

Example for Question 1 – REST API for an Online Shopping Application

A shopping company has both an Android/iOS application and a website. Both clients need to view products, create orders and check order status. The company selects REST because the same lightweight JSON API can be consumed by both platforms.

```
GET /api/products/501
Authorization: Bearer <access-token>

Response:
{
  "productId": 501,
  "name": "Wireless Headphones",
  "price": 2499,
  "stock": 32
}
```

To create an order, the mobile app can send:

```
POST /api/orders
Content-Type: application/json
Authorization: Bearer <access-token>

{
  "productId": 501,
  "quantity": 1,
  "deliveryType": "standard"
}

Response:
{
  "orderId": "ORD-10045",
  "status": "confirmed"
}
```

During a festival sale, additional REST server instances can be started behind the load balancer. Because the API is stateless, a user's first request can be handled by Server A and the next request by Server B. This example demonstrates REST performance, scalability and platform independence. HTTPS, authentication tokens, authorization and rate limiting provide security.

Example for Question 2 – E-Commerce Flash Sale with 50,000 Concurrent Users

Assume an e-commerce platform announces a one-hour flash sale. About 50,000 users simultaneously open the same product page. If every request directly queries one database, the database may become overloaded.

1. Users request the product page through the CDN/WAF.
2. Static images and scripts are served by the CDN without reaching the application server.
3. The load balancer distributes API requests across multiple application servers.
4. Product name, price and availability summary are read from Redis cache whenever possible.
5. Only necessary requests reach the main database.
6. When an order is created, background work such as email and invoice generation is placed on a message queue.
7. Database read replicas can serve read-heavy queries while the primary handles required writes.

Alternative microservices example: Product Service handles catalog requests, Order Service creates orders, Payment Service processes payments and Notification Service sends messages. If payment traffic becomes high, only Payment Service needs extra instances. This improves independent scalability and fault isolation.

Example for Question 3 – Secure ₹5,000 Bank Transfer

A customer uses a mobile banking application to transfer ₹5,000 to a registered beneficiary. The transaction must never be sent over plain HTTP.

```
POST /api/transfers
Authorization: Bearer <short-lived-token>
Idempotency-Key: TRF-2026-10081
Content-Type: application/json

{
 "beneficiaryId": "BEN-204",
 "amount": 5000,
 "currency": "INR"
}
```

The client first establishes an HTTPS/TLS connection and validates the bank server's certificate. After authentication and transaction authorization, the encrypted request reaches the banking API. The server checks account permissions, balance, transaction limits and fraud rules before executing the transfer.

```
{
 "transactionId": "TXN-884021",
```

```
  "status": "SUCCESS",
  "amount": 5000,
  "currency": "INR"
}
```

If the mobile network drops after the request is submitted, the client must not blindly create another transfer. The idempotency key allows the bank to recognize a retry of the same operation and return the existing result instead of debiting the account twice. This illustrates why banking security requires TLS plus application-level authentication, authorization, auditing and safe transaction semantics.

Example for Question 4 – Payment Gateway Integration

Consider an online store where a customer pays ₹1,200. The browser sends the order request to the store backend. The backend validates the order and communicates with the payment provider using credentials stored securely on the server.

Application Backend → Payment Provider

```
POST /payments
Authorization: Bearer <provider-token>
Idempotency-Key: PAY-ORD-7001

{
  "orderId": "ORD-7001",
  "amount": 1200,
  "currency": "INR"
}
```

The provider may initially return a pending response:

```
{
  "paymentId": "PAY-91025",
  "status": "PENDING"
}
```

The application should not mark the order as paid merely because the request was accepted. After payment completion, the gateway can send a signed webhook to the backend. The backend verifies the webhook signature, checks the payment ID, order ID and amount, and then updates the order to PAID.

Alternative design: if the payment provider is temporarily slow, non-interactive reconciliation work can be placed on a message queue. A worker retries temporary failures with exponential backoff. For a Maps API, the same integration pattern can cache repeated geocoding/map results to reduce API calls and improve performance.

Example for Question 5 – Payment Provider Failure and Circuit Breaker

Suppose the checkout service calls an external payment provider, but the provider starts timing out. Without fault tolerance, hundreds of requests may wait until timeout, consuming server threads and producing a poor user experience.

1. Checkout sends a payment request with a strict timeout.
2. The provider fails to respond within the allowed time.
3. The service records the failure and may perform a bounded retry if the operation is safe.
4. After repeated failures, the circuit breaker changes from CLOSED to OPEN.
5. New payment attempts fail quickly instead of repeatedly waiting for the unavailable provider.
6. The application displays a user-safe message such as: 'Payment service is temporarily unavailable. Please try again shortly.'
7. After a recovery interval, the circuit enters HALF-OPEN and allows a small test request.
8. If the provider succeeds, the circuit returns to CLOSED; if it fails, the circuit opens again.
9. Logs, metrics and alerts notify the operations team of the increased failure rate.

Example structured error response:

```
{
  "error": {
    "code": "PAYMENT_SERVICE_UNAVAILABLE",
    "message": "Payment service is temporarily unavailable.",
    "requestId": "REQ-90217"
  }
}
```

Another example is a recommendation service failure. Instead of making the entire website unavailable, the application can temporarily hide personalized recommendations and continue showing product search, cart and checkout. This is called graceful degradation and helps maintain a smooth user experience during partial failures.